Unidad 3 / Escenario 5

Lectura fundamental

Contenedores estándar de datos

Contenido

1 Listas y Arreglos 1

2 Pilas 3

3 Colas 4

4 Colas de prioridad 5

5 Problemas propuestos 6

Palabras clave: List, ArrayList, LinkedList, Stack, Queue

En el Escenario anterior se trataron las maneras primitivas de los lenguajes de programación de almacenar datos del mismo tipo: arreglos de una o más dimensiones. También pudo conocer y practicar el diseño e implementación de métodos en Java. Ahora bien, con este documento familiarícese con los principales contenedores estándar de datos lineales. Aunque todo lo que se puede realizar con estos contenedores lineales se puede hacer con arreglos de tipo nativo de tamaño fijo, estos contenedores ofrecen muy buenas prestaciones respecto a su complejidad computacional y facilidades de uso.

1. Listas y Arreglos

Los primeros contenedores, y tal vez los más usados, son los de tipo arreglo o lista. Sus comportamientos (métodos estándar disponibles) son generalmente los mismos en todos los lenguajes de programación de propósito general. A estos contenedores se accede de la misma manera como con los arreglos de tamaño fijo: con un índice. Tienen la ventaja de que su tamaño es dinámico; inicialmente, al crearlos (instanciarlos) están vacíos, y se puede ir agregando elementos al final, al inicio, e incluso en posiciones aleatorias intermedias.

En Java y en muchos lenguajes de programación hay generalmente varias implementaciones de listas lineales de acceso aleatorio. En el caso de Java se tienen los *ArrayList* (Oracle, 1999a), los *LinkedList* (Oracle, 1999b) y los *Vector* (Oracle, 1999e). Los *ArrayList* funcionan como una lista, aunque su implementación interna es mediante arreglos de tamaño fijo. Los *LinkedList* tienen una implementación interna mediante listas enlazadas. Finalmente, los *Vector* son listas sincronizadas que están protegidas para programas de múltiples hilos. La discusión detallada de las diferencias en cuanto a complejidades de las operaciones disponibles y limitaciones de cada una de estas implementaciones (Satck Overflow, 2018) está por fuera del objetivo de este módulo, pero su información puede llegar a ser importante para la selección de cuál de estas tres implementaciones se debe hacer.

En el programa mostrado en el Código 1 se pueden ver los principales métodos que tienen los programadores a su disposición en la clase *ArrayList*. Usted debe, como en todos los escenarios, probar el programa con diferentes entradas. El programa lee primero un número entero positivo *n*, seguido de *n* números enteros. Luego, prueba varios métodos útiles de la clase *ArrayList*, como lo son: agregar un elemento, buscar un elemento y recuperar el primer índice en el que está, recuperar el último índice en el que está, eliminar un índice, obtener el elemento en un índice, ordenar el contenido del *ArrayList* de manera ascendente usando el método sort de la clase *Collections*.

Todas las clases de contenedores estándar que se abordan en este documento son de tipo *template*. Esto quiere decir que se pueden usar para almacenar elementos de cualquier tipo de datos; en particular, el ejemplo del código 1 y los demás ejemplos muestran su uso con enteros, pero se pueden emplear con otros tipos nativos vía sus clases envolventes (por ejemplo, *Double* para los *doubles*), así como para cualquier tipo de dato estándar o definido por el programador. También se pueden utilizar para modelar arreglos de varias dimensiones instanciando una lista de listas; ejemplo: ArrayList¡ArrayList¡Integer¿ ¿ matrix. Usando polimorfismo y herencia, se pueden aplicar estos contenedores para almacenar elementos de diferente tipo.

Otro ejercicio recomendado para hacer con el código 1 es modificarlo para que, en lugar de utilizar los *ArrayList*, se usen los *LinkedList* y también los *Vector*. El cambio es relativamente fácil puesto que los métodos de las tres clases son casi los mismos. Queda como ejercicio para el lector investigar a profundidad las diferencias entre los tres tipos de contenedores lineales mencionados en esta sección.

```
import java.util.ArrayList; import
java.util.Collections; import
java.util.Scanner; public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        ArrayList<Integer> list = new ArrayList<Integer>();
        System.out.print("How many initial integers? ");
        int n = sn.nextInt();
        for( int i = 0; i < n; ++ i ){
            System.out.print("Type an integer: ");
            int tmp = sn.nextInt();
            list.add(tmp);
        }

        System.out.println("List printed as String: ");
        System.out.println(list);

        System.out.println("List traversed by a for-each loop: ");
        for( int k: list )
            System.out.print(k+" ");
        System.out.println();

        System.out.println("List traversed by a for loop: ");
        for( int i = 0; i < list.size(); ++ i )
            System.out.print(list.get(i)+" ");
        System.out.println();

        System.out.print("Type an integer to search: ");
        int q = sn.nextInt();
        boolean isIn = list.contains(q);
        int firstIndex = list.indexOf(q);
        int lastIndex = list.lastIndexOf(q);
        System.out.println(q +(isIn?" is":" is not")+" on the list: "+list  );
        System.out.println(q +" is first on index "+firstIndex+" on the list: "+list  );
        System.out.println(q +" is last on index "+lastIndex+" on the list: "+list  );

        System.out.print("Type an index to remove: ");
        int r = sn.nextInt();
        list.remove(r);
        System.out.println("List after removing "+r+": "+list  );

        Collections.sort(list);
        System.out.println("Sorted list: "+list);
    }
}
```

Código 1: ejemplo de uso de listas

2. Pilas

Las pilas, como concepto general, son arreglos de elementos que cumplen el principio LIFO, que en inglés quiere decir *Last Input-First Output*, que traduce "último en entrar, primero en salir". Se puede entender como un contenedor que funciona de manera similar a una pila de libros (o cualquier elemento que se pueda apilar), en la cual se pueden agregar libros uno encima de otro y, en cualquier momento, si se desea retirar uno, el más accesible es el último en ser agregado. Este tipo de comportamiento es de uso común en programación, en gran variedad de aplicaciones y algoritmos. Es por esto que prácticamente todos los lenguajes de alto nivel de propósito general poseen, al menos, una estructura de datos que tiene esta clase de comportamientos implementados de una manera eficiente.

En el Código 2 se pueden ver los principales métodos (comportamientos) de la clase *Stack* del paquete *Collections* de Java. Básicamente, con una pila usted puede agregar elementos al tope (*push*), recuperar sin eliminar el último elemento (*peek*) y recuperar eliminando el último elemento de la pila (*pop*). Usted debe ejecutar y seguir paso a paso la ejecución del programa en el código 2 para apropiarse del funcionamiento de la clase y sus principales métodos. Se recomienda revisar la documentación para más métodos de esta clase (Oracle, 1999d).

```
import java.util.Scanner;
import java.util.Stack;
public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        Stack<Integer> stack = new Stack<Integer>(); stack.push(4);
        stack.push(13);
        stack.push(9);
        System.out.println("Original Stack: "+stack);
        System.out.println("stack.peek(): "+stack.peek());
        System.out.println("Size after peek(): "+stack.size());
        System.out.println("Stack after peek(): "+stack);

        System.out.println("stack.pop() "+stack.pop());
        System.out.println("Size after pop(): "+stack.size());
        System.out.println("Stack after pop(): "+stack);
    }
}
```

Código 2: ejemplo de uso de pilas

3. Colas

De manera similar a las pilas, las colas son de uso común en implementaciones de diferentes algoritmos. Ellas deben su nombre al comportamiento FIFO, que en inglés significa *First-Input First Output*, que traduce "primero en entrar, primero en salir". Esto es básicamente el comportamiento de, por ejemplo, los clientes en un supermercado o automóviles en un peaje, en donde el primero que llega es el primero que sale; es decir, en donde se atienden en orden de llegada. Para este comportamiento, los lenguajes de programación de alto nivel de propósito general tienen, al menos, una implementación eficiente. En el caso de Java es la interfaz *Queue* y *Deque*.

A diferencia de las pilas, que son una clase, las colas, al ser una interfaz, no se pueden instanciar, lo que significa que no se puede hacer *new Queue()*. Por ello, una variable declarada de tipo *Queue* debe ser instanciada en una clase concreta de varias posibles, como la clase *LinkedList*. La elección de la clase concreta con que se instancia una variable declarada de tipo *Queue* define tanto las operaciones disponibles como las complejidades computacionales de ellas. Para más detalle en las opciones de clases concretas para instanciar una variable tipo *Queue*, revise la sección *All Known Implementing Classes* en la documentación de Java (Oracle, 1999c). En el Código 3 se pueden ver en uso algunos de los principales métodos disponibles para la interfaz *Queue* instanciada como una *LinkedList*. Usted debe, como siempre, ejecutar y hacer seguimiento paso a paso del código. Se recomienda también probar la instanciación con clases diferentes como *ArrayDeque* y ver qué otros comportamientos se tienen a disposición.

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;
public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        Queue<Integer> queue = new LinkedList<Integer>();
        queue.add(4);
        queue.add(13);
        queue.add(9);
        System.out.println("Original Queue: "+queue);
        System.out.println("Queue.peek: "+queue.peek());
        System.out.println("Size after peek(): "+queue.size());
        System.out.println("Queue after peek(): "+queue);

        System.out.println("queue.poll(): "+queue.poll());
        System.out.println("Size after pop(): "+queue.size());
        System.out.println("Queue after pop(): "+queue);
    }
}
```

Código 3: ejemplo de uso de colas

4. Colas de prioridad

En muchas ocasiones, en implementaciones de algoritmos es necesario entender o procesar tareas de una manera diferente a las aproximaciones tipo FIFO y LIFO, vistas en secciones anteriores, y que se resuelven fácilmente con colas y pilas respectivamente. Una variación de las colas tradicionales tipo FIFO son las colas de prioridad. En una estructura de datos de este tipo se eliminan (atienden) elementos según una prioridad dada.

En Java, la clase concreta *PriorityQueue* ofrece la posibilidad de comportamiento en el cual los datos en la estructura de datos se eliminan de acuerdo con el orden establecido de los elementos En el Código 4 se pueden ver en funcionamiento los principales métodos de la clase *PriorityQueue*. En este ejemplo, de manera estándar y por defecto, se le da prioridad a los números menores a la hora de ser atendidos; es decir, cuando se invocan los métodos *peek* o *poll*. Se recomienda hacer un seguimiento paso a paso de este ejercicio.

```
import java.util.PriorityQueue;
import java.util.Queue;
import java.util.Scanner;
public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        Queue<Integer> queue = new PriorityQueue<Integer>();
        queue.add(4);
        queue.add(13);
        queue.add(9);
        queue.add(1);
        queue.add(-2);
        queue.add(90);
        System.out.println("Original Queue: "+queue);
        System.out.println("Queue.peek: "+queue.peek());
        System.out.println("Size after peek(): "+queue.size());
        System.out.println("Queue after peek(): "+queue);

        System.out.println("queue.poll(): "+queue.poll());
        System.out.println("Size after pop(): "+queue.size());
        System.out.println("Queue after pop(): "+queue);
    }
}
```

Código 4: ejemplo de uso de colas de prioridad

5. Problemas propuestos

Casi todos problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java que tome como entrada ciertos datos leídos desde la entrada estándar del programa y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con los datos de entrada. En todos los problemas propuestos se debe asumir que las entradas cumplen con lo estipulado en la sección "**`Descripción de entrada`**" de cada uno. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo estipulado en la sección "**`Descripción de salida`**", y que sea únicamente lo solicitado allí, nada más.

Teniendo en cuenta que en este Escenario se expone sobre contenedores lineales, se espera que en todas las soluciones se usen de alguna manera. Como requisito adicional, toda la lectura de datos (uso de *Scanner*) y toda impresión de datos a la salida estándar (uso de *System.out*) debe hacerse en el método *main*; es decir, en los demás métodos no puede haber ni lectura ni impresión de datos.

Aunque todos los ejercicios de este Escenario se pueden resolver usando arreglos de tamaño fijo y también usando colecciones estándar no lineales del lenguaje, como *TreeSet* y *TreeMap* (y es posible que usted conozca esta estructura), parte del ejercicio aquí es intentar resolver el problema únicamente con estructuras de datos lineales. Posteriormente, se proponen estos ejercicios para ser resueltos con contenedores estándar no lineales de Java.

1. Hacer un seguimiento paso a paso para el programa en el código 1 con la entrada: 5 2 2 2 3 3 3 3. Verificar su trabajo ejecutando el programa e ingresando los datos mencionados.

2. Pilas de libros: diseñe e implemente un programa en Java que simule el comportamiento de poner libros en dos pilas y moverlos entre ellas.

- Descripción de entrada: un número entero n entre 0 y 10000, seguido de n líneas, en las cuales cada una es una operación del tipo: agregar a la pila A, agregar a la pila B, remover de pila A, remover de pila, pasar de pila A a la pila B, pasar de la pila B a la pila A. Los elementos a agregar/remover/cambiar de pila serán representados por cadenas sin espacios ni tabulaciones. Las instrucciones pueden ser una de las siguientes:
- pop A: remover elemento de pila A.
- pop B: remover elemento de pila B.
- add XXX to A: agregar elemento a pila A.
- add XXX to B: agregar elemento a pila B.
- pop from A push in B: eliminar elemento de pila A y agregarlo a pila B. En caso de que A esté vacía, no hacer nada.
- pop from B push in A: eliminar elemento de pila B y agregarlo a pila A. En caso de que B esté vacía, no hacer nada.

- Descripción de salida: dos líneas, una con el contenido de la pila A y otra con el contenido de la pila B, separado por espacios
- Ejemplo de entrada:
  add book1 to A add book2 to A add book3 to A add book4 to B pop from A push in B pop from A push in B pop B
- Ejemplo de salida:
  book1 book3 book4

3. Contador de dígitos: diseñe e implemente un programa en Java que responda la siguiente pregunta: si quisiera escribir con lápiz y papel los números enteros desde el 0 hasta el *n* (*n* es un argumento que lee el programa), ¿cuántas veces escribiría cada dígito del 0 al 9?

- Descripción de entrada: un número entero entre 0 y 10000.
- Descripción de salida: 10 líneas, una para cada dígito del 0 al 9, indicando cuántas veces se escribiría el dígito al escribir los números del 0 al *n*.
- Ejemplo de entrada:
  12
- Ejemplo de salida:
  El dígito 0 se escribiría 2 veces.
  El dígito 1 se escribiría 5 veces.
  El dígito 2 se escribiría 2 veces.
  El dígito 3 se escribiría 1 veces.
  El dígito 4 se escribiría 1 veces.
  El dígito 5 se escribiría 1 veces.
  El dígito 6 se escribiría 1 veces.
  El dígito 7 se escribiría 1 veces.
  El dígito 8 se escribiría 1 veces.
  El dígito 9 se escribiría 1 veces.

4. Centro de masas: dado un conjunto de puntos con masa en el plano, hacer un programa que lea estos datos y calcule y muestre el centro de masas de estos puntos. El centro de masas está dado por la siguiente fórmula:

$$x_{cm} = \frac{\sum_{i=1}^{n} m_i * x_i}{\sum_{i=1}^{n} m_i}$$

$$y_{cm} = \frac{\sum_{i=1}^{n} m_i * y_i}{\sum_{i=1}^{n} m_i}$$

- Descripción de entrada: un número entero *n* seguido de *n* triplas de números. Cada tripla es un punto con su masa, primero la coordenada *x*, luego la coordenada *y* y luego la masa *m* para ese punto.
- Descripción de salida: dos números, la coordenada *x* del centro de masa y la coordenada *y* del centro de masa de los puntos leídos.
- Ejemplo de entrada: 5
  10 10 2
  10 20 4
  10 30 8
  10 40 1
  10 50 16
- Ejemplo de salida:
  10 38.065

5. Espectro: diseñar e implementar un programa que tome como entrada 5 números enteros positivos y produzca como salida un patrón, como se muestra en el ejemplo.
   - Descripción de entrada: un número *n* entero positivo seguido de n números enteros no negativos. Todos
   - entre 0 y 100.
   - Descripción de salida: el patrón según se muestra en el ejemplo. El primer número leído n será la cantidad de columnas de la matriz a imprimir. Los siguientes n números serán la cantidad de numerales que salen en cada columna. Así, en el ejemplo, en la primera columna hay 3 numerales, en la segunda un numeral, en la tercera 8 numerales (el mayor valor da la cantidad de filas), la penúltima columna muestra 4 numerales y la última 2 numerales.
   - Ejemplo de entrada: 5
     3 1 8 4 2
   - Ejemplo de salida:

```
..#..
..#..
..#..
..#..
..#..
..##.
#.##.
#.###
#####
```

Código 5: patrón a imprimir en el problema

6. Palíndromas: diseñe un programa que lea como entrada una cadena de texto e imprima y responda la pregunta ¿cuántas subcadenas son palíndromas de longitud? Al menos 2 tiene la cadena leída. El programa debe mostrar todas las subcadenas palíndromas solicitadas en orden lexicográfico y la cantidad de ellas.
   - Descripción de entrada: una cadena de texto.
   - Descripción de salida: las cadenas de texto según lo descrito en el enunciado, de a una por línea y el número indicando la cantidad de cadenas mostradas en una línea separada.

- Ejemplo de entrada:
  anitalavalatinaanaeslamejor
- Ejemplo de salida:
  aa
  ala
  ala
  alavala
  ana
  anitalavalatina
  ava
  italavalati
  laval
  naan
  nitalavalatin
  talavalat
  12

7. Diseñar e implementar un programa que tome como entrada un número entero ***n*** y, seguido a esto, que lea ***n*** números enteros. El programa debe mostrar los números leídos sin repeticiones y ordenados ascendentemente.
   - Descripción de entrada: un número ***n*** entero, seguido de *n* números enteros, todos mayores o iguales que 0 y menores o iguales a 1000.
   - Descripción de salida: los elementos leídos, sin repeticiones y ordenados ascendentemente.
   - Ejemplo de entrada:
     14
     10 29 5 6 29 10 10 10 6 6 6 5 5 5
   - Ejemplo de salida:
     5 6 10 29
8. Diseñar e implementar un programa que tome como entrada un número entero ***n*** y, seguido a esto, lea ***n*** números enteros y diga cuántas veces está cada entero leído en la lista de enteros leídos. Adicionalmente, debe mostrar los números ordenados por frecuencia, es decir, debe primero mostrar el que más veces aparece en la entrada, después el segundo más común, luego el tercero y así sucesivamente.
   - Descripción de entrada: un número ***n*** entero, seguido de ***n*** números enteros, todos mayores o iguales que 0 y menores o iguales a 1000.
   - Descripción de salida: la cantidad de veces que aparece cada número. Los números deben estar en orden descendente respecto a la cantidad de veces que se presenta.
   - Ejemplo de entrada: 13
     10 29 5 6 29 10 2 2 2 10 2 10 2
   - Ejemplo de salida:
     El 2 aparece 5 veces.
     El 10 aparece 4 veces.
     El 29 aparece 2 veces.
     El 6 aparece una vez.
     El 5 aparece una vez.
9. Frecuencias: diseñar e implementar un programa que tome como entrada un número entero **n**; luego, lea n números enteros y después muestre una tabla de frecuencias de los números leídos usando asteriscos *. Deben estar ordenados en la salida respecto a la frecuencia, estando primero los que más veces aparecen en la entrada.

- Descripción de entrada: un número ***n*** entero, seguido de ***n*** números enteros, todos mayores o iguales que 0 y menores o iguales a 1000.
- Descripción de salida: la tabla de asteriscos según el ejemplo.
- Ejemplo de entrada:
  13
  10 29 5 6 29 10 2 2 2 10 2 10 2
- Ejemplo de salida:
- 2 *****
  10 ****
  29 **
  6 *
  5 *

10. Frecuencias: diseñar e implementar un programa que tome como entrada un número entero ***n***; seguido a esto lea *n* números enteros; después, muestre una tabla de frecuencias de los números leídos usando asteriscos *****. Deben estar ordenados en la salida respecto a la frecuencia. La cantidad de asteriscos mostrados por número debe ser la mínima posible, siempre y cuando se conserve la proporción en relación con la cantidad de números leídos. Así, en la entrada del ejemplo, que se muestra abajo para este caso, aparecen 4 asteriscos para el número 10, aunque el número 10 aparece 12 veces. Esto porque la cantidad de asteriscos por número es proporcional a la cantidad de veces que cada número se leyó en la entrada. En este ejemplo, la cantidad de números es 3 multiplicado por la cantidad de asteriscos: 12 veces el número 10, 6 veces el número 29, tres veces el número 6 y 3 veces el número 5 (12+6+3+3=24).
    - Descripción de entrada: un número *n* entero, seguido de *n* números enteros, todos mayores o iguales que 0 y menores o iguales a 1000.
    - Descripción de salida: la tabla de asteriscos según el ejemplo.
    - Ejemplo de entrada:
      24
      10 29 5 6 29 10 10 10 10 29 5 6 29 10 10 10 10 29 5 6 29 10 10 10
    - Ejemplo de salida:
      10 ****
      29 **
      6 *
      5 *
11. Se desea hacer un programa que tome como entrada los datos de estudiantes (nombres completos, números de documento) y una serie de notas, y produzca una lista Estudiante-Promedio, ordenada de mejor a peor promedio de notas, y muestre el resultado a manera de tabla con tres columnas: nombre, cédula y promedio. El formato del archivo de entrada es el siguiente:
    - En la primera línea del archivo de entrada habrá un número entero positivo *n*; este es la cantidad de líneas que siguen y cada línea representa la información de un estudiante. Cada línea debe tener el siguiente
    - formato: nombre completo - número de documento - serie de notas entre 0 y 5 separadas por espacios.
    - Descripción de salida: la lista según lo solicitado.
    - Ejemplo de entrada:
      4 Rafael Alberto Velasquez-889993893-4 3 4 4.5 4 2 1

Jose Figueroa-2334234-4 3 1 3 5
Diego Castilla-232234-3 4 1 2 3 3 2 1 2.3 2.3
Alberto Baron-234213-3 3 1

- Ejemplo de salida:
  Alberto Baron 234213 4.33333
  Rafael Alberto Velasquez 889993893 3.21429
  Jose Figueroa 2334234 3.2
  Diego Castilla 232234 2.36

12.Número de Fibonacci: para este programa se recomienda implementar la función recursiva y optimizar usando *memoization*:

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-1) + f(n-2) & \text{en otro caso.} \end{cases}$$

- Descripción de entrada: un número entero positivo entre 0 y 90.
- Descripción de salida: un entero positivo, el enésimo número de Fibonacci (The OEIS Foundation, 2018).
- Ejemplo de entrada:
  0
- Ejemplo de salida:
  1
- Ejemplo de entrada 2:
  1
- Ejemplo de salida 2:
  1
- Ejemplo de entrada 3:
  2
- Ejemplo de salida 3:
  2
- Ejemplo de entrada 4:
  40
- Ejemplo de salida 4:
  102334155

Referencias

Oracle. (1999a). *Class ArrayList* <E>. Recuperado de https://docs.oracle. com/javase/7/docs/api/java/util/ArrayList.html

Oracle. (1999b). *Class LinkedList* <E>. Recuperado de https://docs. oracle.com/javase/7/docs/api/java/util/LinkedList.html

Oracle. (1999c). *Interface Queue* <E>. Recuperado de https://docs.oracle. com/javase/7/docs/api/java/util/Queue.html

Oracle. (1999d). *Class Stack* <E>. Recuperado de https://docs.oracle. com/javase/7/docs/api/java/util/Stack.html

Oracle. (1999e). *Class Vector* <E>. Recuperado de https://docs.oracle. com/javase/7/docs/api/java/util/Vector.html

Stack Overflow. (2018). *Big-O Summary for Java Collection Framework implementations?* Recuperado de https://stackoverflow. com/ questions/ 559839 / big- o- summary- for - java- collections- framework-implementations

The OEIS Foundation. (2018). *The On-Line Encyclopedia of Integer Sequences: Fibonacci Numbers.* Recuperado de https://oeis.org/A000045

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 3: Principales contenedores de datos
Escenario 5: Contenedores estándar de datos

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador Gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.